Open source tools for stream and batch processing

Data-driven organisations are looking at improving and integrating real-time data costs and methods. At the centre of it all is an open source ecosystem of tools that allows a business access to stream and batch processing capabilities.

For batch processing, Apache Hadoop and Apache Spark are well known for scaling processing in a scheduled manner over large datasets. Apache Airflow and Luigi are orchestration tools that manage complex workflow processes. These can be processed in various settings such as for generating daily reports, transforming data warehouses and training machine learning models.

When immediate outcomes matter (fraud detection, real-time recommendations, etc), live streaming is best. Tools like Apache Kafka can stream high throughput data, while tools like Apache Flink and Kafka Streams can apply transformations with milliseconds or less latency and provide reliable data. Debezium is a game changer here — it can efficiently replicate data from a source database to another system capturing all the changes in a streaming fashion while keeping different systems in sync in real time.

Several modern tools, such as Apache Beam and Delta Lake, offer unified functionality, allowing developers to design pipelines that can consume from streams and batches interchangeably. This enables hybrid architectures to utilise real-time speed while relying on historical intelligence.

Your choice of tool will depend on latency, volume of data, data quality, team experience, and many other factors. Most organisations benefit from a combination—streaming for real-time and batch processing for historical and long-term analysis. The good news is that open source tools are very interoperable, easily allowing you to design a pipeline suitable for your business needs.

In a world where we are expected to not just deliver insights faster, but to also make smarter decisions, open source data tools are no longer optional, but a necessity.

What is special about stream processing?

In the fast-paced AI world, stream processing has become mandatory for organisations that need real-time insights and responsiveness. Streaming enables organisations to compute continuously, generate real-time alerts, and react immediately as events arise.

Streaming begins with the notion of an event-driven architecture. Each data event (including sensor readings, customer clicks, and transactions) needs to be processed as soon as it occurs. This means that streaming systems need to maintain state, manage out-of-order events, and support time windows while also demonstrating scale and fault tolerance.

The biggest advantages of stream processing are low latency consumption and that organisations can act in milliseconds. It enables fraud detection, live personalisation, and operational monitoring. The challenges are that these systems are difficult to build and maintain, and they operate under uncertain conditions. Keeping track of the exactly-once processing of all messages, handling failures, and managing state consistency are some other challenges.

Premier open source stream processing frameworks are:

Apache Kafka: A distributed event streaming platform and the backbone of many pipelines.

Apache Flink: A sophisticated stream processor with strong support for state, event-time, and windows.

Spark streaming: A micro-batch engine for real-time pipelines with familiar Spark APIs.

Redpanda: A Kafka-compatible event streaming solution with built-in C++ for high-performance streaming speed and ease of use.

When to choose batch processing over stream processing

Utilise batch processing when:

- Your data is mostly static or slow-changing
- Timeliness is not of the essence (i.e., daily sales reports)
- You are looking for the simplest architecture with lowest operational overhead
- The volume of data is huge but real-time insight is unimportant

Industries such as finance (monthly statements), retail (inventory reports), and education (semester records) are the best use cases.

You can think of a batch as filling a water tank — you gather a large volume of data and then process it all at once. Stream is like running water from a tap — the data flows continuously, and you act on it as soon as it arrives.

This simple analogy illustrates the key differences between batch and stream processing:

Batch = Delay + Volume

Stream = Speed + Continuity

Hybrid models: When both worlds meet

The increased complexity of data has fuelled the emergence of hybrid data architectures where both batch and stream processing coexist in the same data architecture. These hybrid data architectures provide users with the low-latency responsive capabilities of a streaming model, while also providing reliability and depth of analysis common in batch analytics models, without sacrificing the business value of either model.

Hybrid architectures are designed to both ingest, process, and serve data in real time and to analyse data historically. These mix batch and stream processing in an intelligent fashion to serve a range of diverse business use cases, including real-time alerts as well as end-of-day reports.